

虚函数与多态

虚函数

在类的定义中，前面有 `virtual` 关键字的成员函数（`virtual`只需要在声明时添加）

在编译时不确定调用哪一个，在运行时才确定

派生类中与基类中虚函数同名同参数表的函数，不加 `virtual` 也自动成为虚函数

构造函数与析构函数中调用虚函数——不是多态

1. 在基类构造函数中调用虚函数，调用的是**基类自己的虚函数**，不会调用派生类重写的版本。
2. 在基类析构函数中调用虚函数，调用的也仍然是**基类自己的虚函数**，不会调用派生类重写的版本。
3. 析构函数**自身**可以是虚函数

多态

在**指针和引用**中起效，对**传值**的函数调用**不起效**

虚函数与指针

通过基类指针调用基类与派生类的**同名虚函数**时：

1. 指针指向**基类**，调用**基类**的虚函数
2. 指针指向**派生类**，调用**派生类**的虚函数

```
#include <iostream>
using namespace std;

// 基类
class Animal {
public:
    virtual void speak() { // 虚函数
        cout << "Animal::speak()" << endl;
    }
};

// 派生类
class Dog : public Animal {
public:
    void speak() { // 重写虚函数
```

```

        cout << "Dog::speak() 汪汪汪! " << endl;
    }
};

// 派生类
class Cat : public Animal {
public:
    void speak() { // 重写虚函数
        cout << "Cat::speak() 喵喵喵! " << endl;
    }
};

int main() {
    Animal animal;
    Dog dog;
    Cat cat;

    Animal* p; // 基类指针

    // 1. 指向基类对象，调用基类的虚函数
    p = &animal;
    p->speak(); // Animal::speak()

    // 2. 指向派生类对象，调用派生类的虚函数
    p = &dog;
    p->speak(); // Dog::speak()

    p = &cat;
    p->speak(); // Cat::speak()

    return 0;
}

```

虚函数与引用

通过**基类引用**调用基类与派生类的**同名虚函数**时：

1. 引用**基类**对象，调用**基类**的虚函数
2. 引用**派生类**对象，调用**派生类**的虚函数

```

#include <iostream>
using namespace std;

```

```

// 基类
class Animal {
public:
    virtual void speak() { // 虚函数
        cout << "Animal::speak()" << endl;
    }
};

// 派生类
class Dog : public Animal {
public:
    void speak() { // 重写虚函数
        cout << "Dog::speak() 汪汪汪! " << endl;
    }
};

// 派生类
class Cat : public Animal {
public:
    void speak() { // 重写虚函数
        cout << "Cat::speak() 喵喵喵! " << endl;
    }
};

// 使用基类引用作为参数
void makeSpeak(Animal& ref) {
    ref.speak(); // 根据实际引用的对象调用相应的虚函数
}

int main() {
    Animal animal;
    Dog dog;
    Cat cat;

    // 1. 引用基类对象，调用基类的虚函数
    Animal& ref1 = animal;
    ref1.speak(); // Animal::speak()

    // 2. 引用派生类对象，调用派生类的虚函数
    Animal& ref2 = dog;
    ref2.speak(); // Dog::speak()

    Animal& ref3 = cat;

```

```

ref3.speak(); // Cat::speak()

// 通过函数参数传递引用
cout << "\n==== 通过函数调用 =====> endl;
makeSpeak(animal); // Animal::speak()
makeSpeak(dog);    // Dog::speak()
makeSpeak(cat);    // Cat::speak()

return 0;
}

```

动态连编

虚函数表：每一个有虚函数的类（or 有虚函数的类的派生类）都有一个**虚函数表**，该类的任何对象中都放着虚函数表的指针。虚函数表中列出了该类的虚函数地址。

有虚函数的类的size会增加4个字节，存的就是虚函数表的地址

虚析构函数

问题：通过基类的指针删除派生类对象时，通常只调用基类的析构函数

解决方法：把**基类**的析构函数声明为**virtual**

一般来说，一个类如果定义了虚函数or打算作为基类使用，则应该将析构函数定义为虚函数

虚函数不允许作为构造函数

```

#include <iostream>
using namespace std;

// ===== 基类 =====
class Animal {
public:
    Animal() {
        cout << "Animal 构造函数" << endl;
    }

    // ✅ 虚析构函数
    virtual ~Animal() {
        cout << "Animal 析构函数" << endl;
    }
};

```

```
// ===== 派生类 =====
class Dog : public Animal {
private:
    int* data; // 模拟动态资源

public:
    Dog() {
        data = new int[100];
        cout << "Dog 构造函数（分配了内存）" << endl;
    }

    ~Dog() override {
        delete[] data;
        cout << "Dog 析构函数（释放了内存）" << endl;
    }
};

// ===== 主函数 =====
int main() {
    cout << "==== 通过基类指针删除派生类对象 =====" << endl;
    Animal* ptr = new Dog(); // 基类指针指向派生类对象
    delete ptr;              // 触发多态析构

    return 0;
}
```

纯虚函数与抽象类

定义：

纯虚函数：**没有函数体**的虚函数

```
virtual void show() = 0;
```

`= 0` 表示这个函数没有具体实现，要求派生类必须重写。

抽象类：包含**纯虚函数**的类

1. 抽象类只能作为基类来派生新类，**不能创建独立的抽象类的对象**
2. 抽象类的指针和引用可以指向抽象类**派生出来的类的对象**
3. 在抽象类的**成员函数**可以调用纯虚函数，但是**构造函数**和**析构函数**内部不能调用纯虚函数
4. 如果一个类从抽象类派生而来，那么**当且仅当**它实现了基类里**所有纯虚函数**，它才能成为非抽象类

```
#include <iostream>
using namespace std;

// 抽象类
class Animal {
public:
    virtual void speak() = 0;    // 纯虚函数
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "狗在叫: 汪汪! " << endl;
    }
};

class Cat : public Animal {
public:
    void speak() override {
        cout << "猫在叫: 喵喵! " << endl;
    }
};

int main() {
    // Animal a;    // 错误: 抽象类不能实例化

    Animal* p1 = new Dog();
    Animal* p2 = new Cat();

    p1->speak();
    p2->speak();

    delete p1;
    delete p2;

    return 0;
}
```